# Database Design — mediciones

*Diseño lógico de la base de datos del Sprint (servidor REST + web). Notación: Logical Design & Reverse Engineering Specification v3.*

## Component Design

La base de datos es MySQL (XAMPP) y contiene una única tabla, `mediciones`, que guarda una fila por cada medición que la app Android sube tras detectar el beacon de la plaquita. Gestión con phpMyAdmin.

```
================================================================================
==
TABLE: mediciones
DESCRIPTION: Guarda cada medición (tipo y valor) que la app Android sube al
servidor tras detectar el beacon.

COLUMNS:

+ id_plaquita | VARCHAR(20)  | NOT NULL |
+ tipo_medicion  | VARCHAR(20)  | NOT NULL |
+ valor_medicion | SMALLINT     | NOT NULL |
+ fecha          | DATETIME     | NOT NULL | CURRENT_TIMESTAMP

PRIMARY KEY: (id_plaquita, fecha)

FOREIGN KEYS:

+ (ninguna)

CONSTRAINTS:

+ tipo_medicion solo admite "CO2", "temperatura" o "prueba" (regla de negocio RN1;
la valida la capa de negocio)
+ valor_medicion es obligatorio y entero (regla de negocio RN2)
+ fecha la rellena la propia base de datos (DEFAULT CURRENT_TIMESTAMP); nunca
llega del cliente (regla de negocio RN3)
================================================================================
==
```

### Correspondencia lógica → SQL

| Campo lógico | Tipo lógico | Tipo SQL (MySQL, vía phpMyAdmin) | Notas |
|---|---|---|---|
| `id_plaquita` | `Text` | `VARCHAR(20)` | NOT NULL — "GTI-3A"; parte de la clave |
| `tipo_medicion` | `Text` | `VARCHAR(20)` | NOT NULL — RN1: `"CO2"` \| `"temperatura"` \| `"prueba"` |

| Campo lógico | Tipo lógico | Tipo SQL (MySQL, vía phpMyAdmin) | Notas |
|---|---|---|---|
| `valor_medicion` | `Z` | `SMALLINT` | NOT NULL — RN2: valor del proyecto (`1234`) |
| `fecha` | `Tiempo` | `DATETIME` | NOT NULL, `DEFAULT CURRENT_TIMESTAMP` — RN3; parte de la clave |

DDL

```
CREATE DATABASE IF NOT EXISTS proyecto_beacon
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE proyecto_beacon;

CREATE TABLE IF NOT EXISTS mediciones (
  id_plaquita VARCHAR(20)  NOT NULL,
  tipo_medicion  VARCHAR(20)  NOT NULL,
  valor_medicion SMALLINT     NOT NULL,
  fecha          DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ( id_plaquita, fecha ),
  INDEX idx_mediciones_fecha ( fecha )
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

## Design Clarifications

- `tipo_medicion` es **texto** (no un código numérico): la app Android traduce el código que viaja en el `major` del iBeacon a texto antes de enviar. Lo que se guarda y se enseña es el nombre del dato.
- `valor_medicion` es `SMALLINT`: el valor del proyecto (`1234`) cabe de sobra en 16 bits con signo.
- El índice `idx_mediciones_fecha` acelera el `ORDER BY fecha DESC LIMIT 50` que usa la web al consultar las últimas mediciones.
- La tabla **no tiene columna `id` artificial**: la clave de cada fila es la natural compuesta `(id_plaquita, fecha)`. La app **no** envía `fecha`: la genera la base de datos (`DEFAULT CURRENT_TIMESTAMP`, RN3). Tampoco se guardan `uuid_beacon`, `major`, `minor`, `tx_power` ni `rssi`.
- No existe tabla de usuarios: subir y consultar mediciones son operaciones públicas (regla de negocio RN4).

## General Rules

- **Programming Language:** SQL (MySQL 8 / MariaDB de XAMPP) para el DDL; el acceso a la base de datos se hace desde PHP con PDO y consultas parametrizadas.
- **Function/Method Headers:** toda operación de acceso a datos (consulta, inserción) se documenta en su cabecera con su diseño lógico dentro de un bloque de comentario delimitado por líneas discontinuas (`----------`).
- **Code Readability:** el DDL y las consultas deben ser claros y explicativos por sí mismos; los comentarios inline se reducen al mínimo.

- **Automated Testing:** el DDL real se prueba con una base de datos desechable (`proyecto_beacon_test`), aplicando el `.sql` de verdad y comprobando el resultado de las operaciones.